

FULL BAYES!

```
#load Rstanarm for bayesian modelling and plot the data#  
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —  
## ✓ dplyr      1.2.1      ✓ readr      2.2.0  
## ✓ forcats    1.0.1      ✓ stringr    1.6.0  
## ✓ ggplot2     4.0.3      ✓ tibble     3.3.1  
## ✓ lubridate  1.9.5      ✓ tidyr      1.3.2  
## ✓ purrr      1.2.2  
## — Conflicts — tidyverse_conflicts() —  
## ✗ dplyr::filter() masks stats::filter()  
## ✗ dplyr::lag()     masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(arm)
```

```
## Loading required package: MASS  
##  
## Attaching package: 'MASS'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     select  
##  
## Loading required package: Matrix  
##  
## Attaching package: 'Matrix'  
##  
## The following objects are masked from 'package:tidyr':  
##  
##     expand, pack, unpack  
##  
## Loading required package: lme4  
##  
## arm (Version 1.15-3, built: 2026-4-15)  
##  
## Working directory is C:/Users/alexa/OneDrive/Documents/(No subject)/Bayesregression
```

```
library(rstanarm)
```

```
## Loading required package: Rcpp  
## This is rstanarm version 2.32.2  
## - See https://mc-stan.org/rstanarm/articles/priors for changes to default priors!  
## - Default priors may change, so it's safest to specify priors, even if equivalent to the defaults.  
## - For execution on a local, multicore CPU with excess RAM we recommend calling  
##   options(mc.cores = parallel::detectCores())  
##  
## Attaching package: 'rstanarm'  
##  
## The following objects are masked from 'package:arm':  
##  
##     invlogit, logit
```

```
library(readxl)
library(lmtest)
```

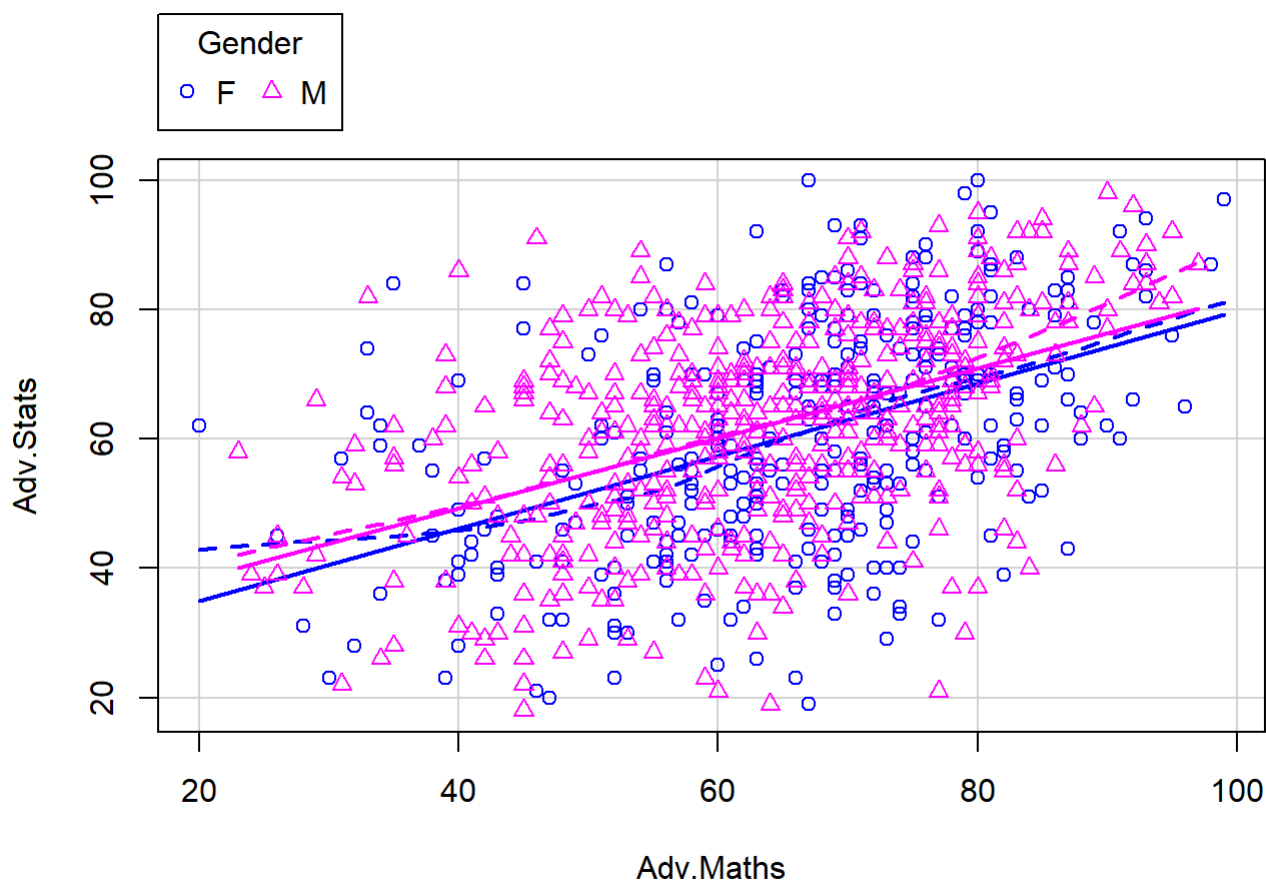
```
## Loading required package: zoo
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
```

```
library(bayestestR)
```

```
##
## Attaching package: 'bayestestR'
##
## The following object is masked from 'package:arm':
##
##   display
```

```
Maths_and_Stats_grades <- read_excel("C:/Users/alexa/Downloads/Maths and Stats grades.xlsx")
car::scatterplot(Adv.Stats ~ Adv.Maths | Gender, data = Maths_and_Stats_grades)
```

```
## Registered S3 method overwritten by 'car':
##   method          from
##   na.action.merMod lme4
```



Technical trivia

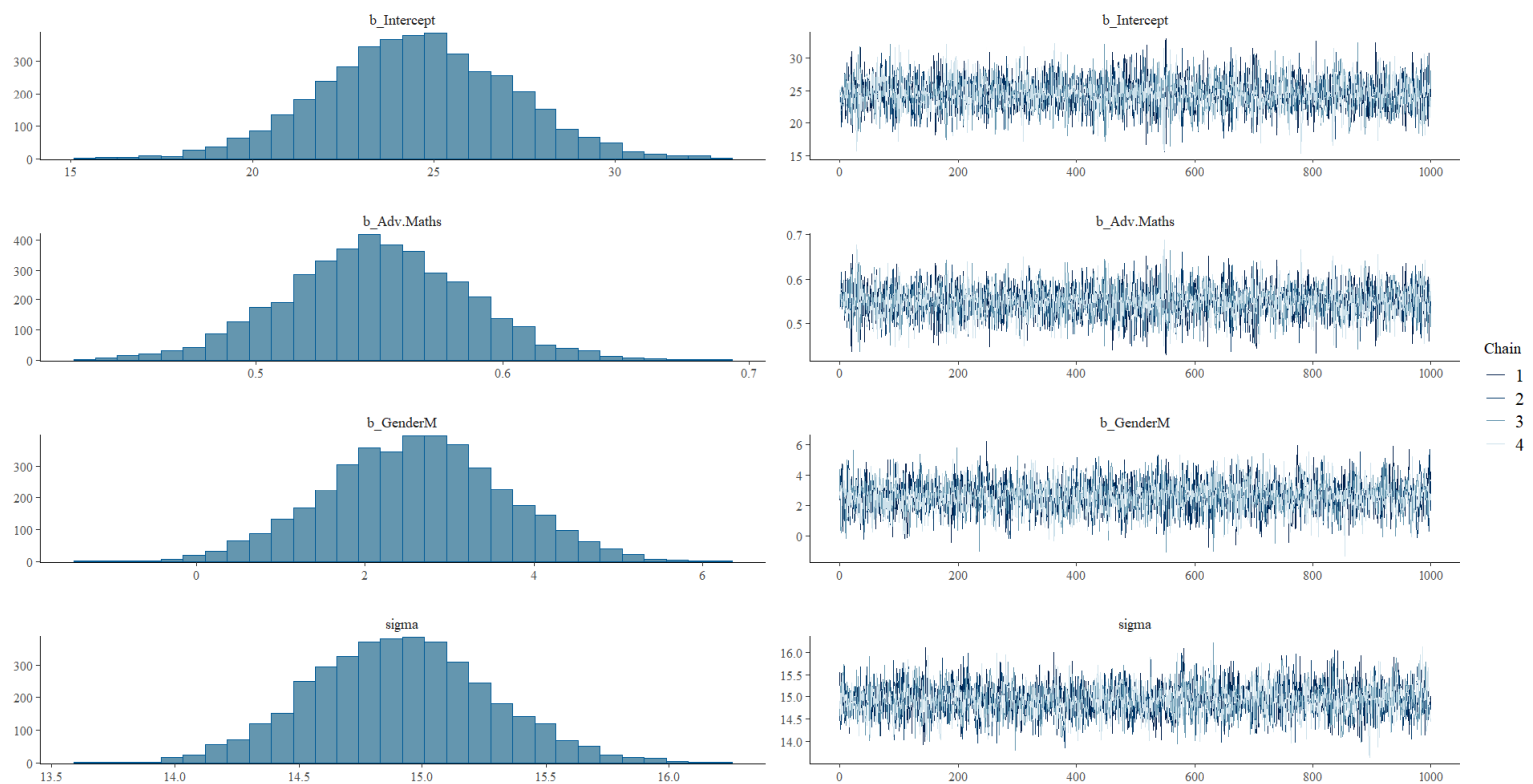
Logged version of bayes rule for parameter conditional on data is

$$\log p(\theta_i | D) = \log p(D | \theta_i) + \log p(\theta_i) - \log \left(\sum_{k=1}^K p(D | \theta_k) p(\theta_k) \right)$$

where the first term is the log likelihood(through which the data effects the posterior and the second term is the log prior distribution representing our belief about the parameter, and probability of data is expanded by law of total probability. One way to think of a Bayesian estimate is as of one which maximizes the log of posterior probability of our parameter given data or simply($p(D)$ is a constant so it drops off)), indeed this is the mode or the peak of the posterior distribution where we could get SE by looking at curvature at the maximum. Note that finding MAP requires balancing both the data pull and the pull of the log prior-if data has high variance or is sparse prior dominates the estimate.

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \{ \log p(D | \theta) + \log p(\theta) \}$$

The Markov Chain Monte Carlo method however bypasses this by simulating loads of independent between each other but dependent within each other(within one step) search paths(random but dependent combinations of plausible coefficients) which leap frog around the discrete posterior and eventually map out its probability density surface until they start producing similar results. This allows us to get the entire posterior distribution , not just the MAP estimate. Throwing sh*t at the Probability density wall until it sticks(converges) hasn't been so effective and literal.



This is actually the simulation paths used for our model, note how there are 4 chains mapping out each parameter and eventually starting to bounce around the same mean/median with the highest posterior probability.

End of technical trivia

Since we know that the relationship between maths and stats grades should be strong and positive we suppose that it should be near 1 to 1 so we set the prior mean of the coefficient to 0.8, we don't really know much about how certain we are about spread of that though so we use the scaled up default prior variance to let data dictate the estimate more easily . We dont know much about gender effect so we set mean prior to 0 and similarly use default scaled up variance prior we run the model and let MCMC do its thing to simulate the posterior. We are basically approximating the entire posterior distribution of Stats scores on predictors.

#Full Bayes we are sure that the relationship between maths and stats is near 1 to 1 positive but allow it to vary#

```
BayesReg <- stan_glm(Adv.Stats ~ Adv.Maths + factor(Gender),family=gaussian(),prior = normal(location = c(0.8,0),autoscale = TRUE),data=Maths_and_Stats_grades)
```

```
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.001664 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 16.64 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 1: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 1: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 1: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 1: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 1: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 1: Iteration:  1001 / 2000 [ 50%] (Sampling)
## Chain 1: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 1: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 1: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 1: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 1: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.05 seconds (Warm-up)
## Chain 1:                0.093 seconds (Sampling)
## Chain 1:                0.143 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 2.3e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.23 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 2: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 2: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 2: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 2: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 2: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 2: Iteration:  1001 / 2000 [ 50%] (Sampling)
## Chain 2: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 2: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 2: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 2: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 2: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.05 seconds (Warm-up)
## Chain 2:                0.104 seconds (Sampling)
## Chain 2:                0.154 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 2e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.2 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [  0%] (Warmup)
```

```

## Chain 3: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 3: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 3: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 3: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 3: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 3: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.044 seconds (Warm-up)
## Chain 3: 0.096 seconds (Sampling)
## Chain 3: 0.14 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 2.1e-05 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.21 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 4: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 4: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 4: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 4: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 4: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 4: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 4: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 4: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.07 seconds (Warm-up)
## Chain 4: 0.101 seconds (Sampling)
## Chain 4: 0.171 seconds (Total)
## Chain 4:

```

After our 4 chains(think of them as search parties) converged we have the entire posterior distribution for our model components given data and priors. Lets examine the prior and posterior. Note how the standard deviation of the priors was scaled up or down depending on the actual spread in the data, in essence the variance of the prior regulates how strongly you believe observed effects can differ from the prior mean, the higher the prior sd the less we restrict the freedom data has in dictating the estimated coefficient. For example a sd of 2.94 with a prior mean of 0.8 means we expect 95 percent of effects to lie between -5.0 and 6.6 so if the software encounters something extreme like 10 or -10 it reacts by shrinking it towards the mean of 0.8 to balance prior with sample evidence. If we set variance to 1 than even less extreme values would be shrunk as the interval decreases to [-0.2,1.8]. In our case however its clear that the data overpowered our prior guess with the posterior estimate for the effect of maths at 0.55 as opposed to 0.8.

```
describe_prior(BayesReg)
```

```

##      Parameter Prior_Distribution Prior_Location Prior_Scale
## 1      (Intercept)              normal      61.44195    42.08488
## 2      Adv.Maths                normal       0.80000     2.93709
## 3 factor(Gender)M              normal       0.00000    85.06115

```

```
describe_posterior(BayesReg,rope_range = c(-0.1, 0.1))
```

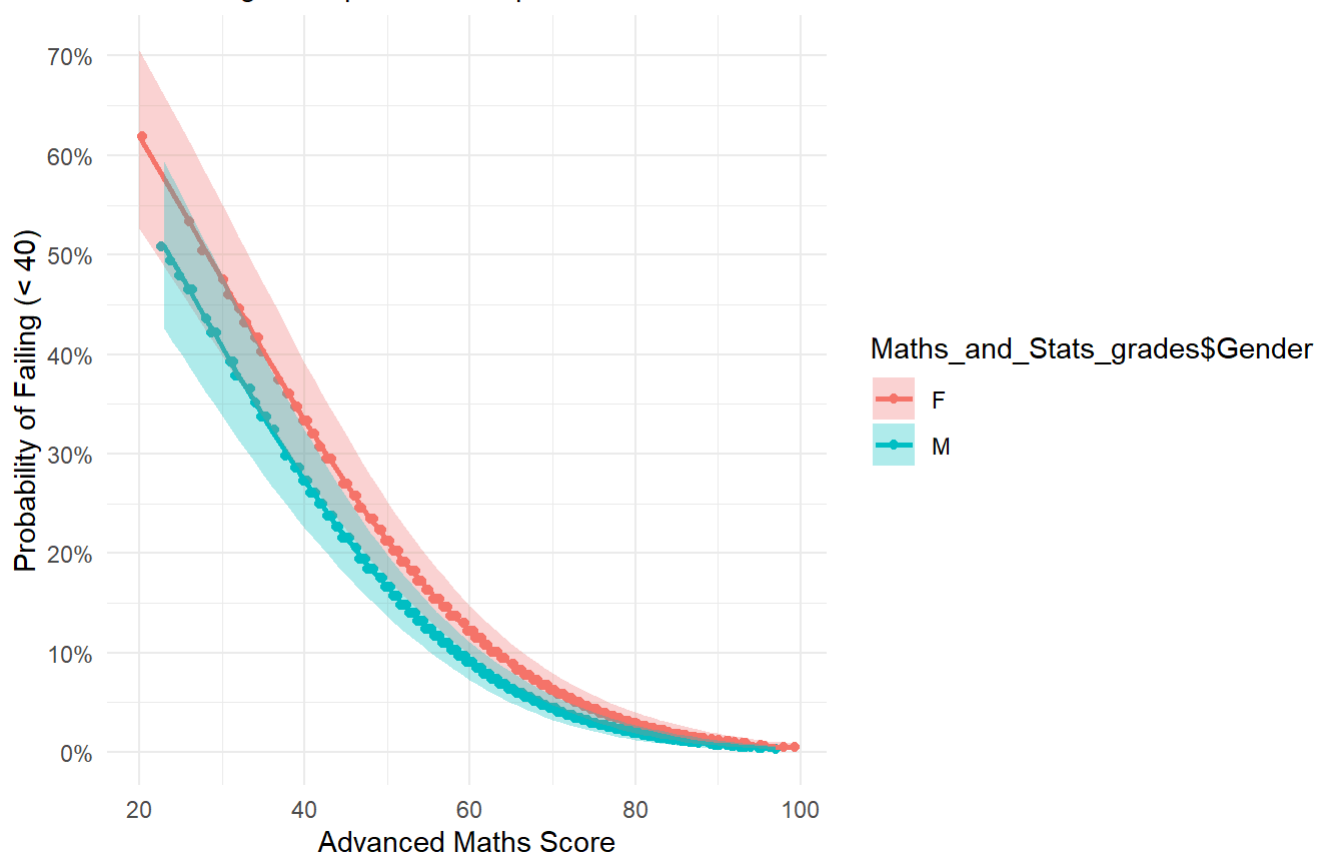
```
## Summary of Posterior Distribution
##
## Parameter      | Median |          95% CI |      pd |          ROPE | % in ROPE
## -----
## (Intercept)    | 24.43 | [19.67, 29.35] | 100% | [-0.10, 0.10] | 0%
## Adv.Maths      | 0.55 | [ 0.48, 0.62] | 100% | [-0.10, 0.10] | 0%
## factor(Gender)M | 2.60 | [ 0.51, 4.65] | 99.50% | [-0.10, 0.10] | 0%
##
## Parameter      | Rhat | ESS (tail)
## -----
## (Intercept)    | 1.000 | 2840
## Adv.Maths      | 1.000 | 2745
## factor(Gender)M | 1.000 | 2943
```

PD here stands for Probability of Direction i.e its the posterior probability that the parameter positive or negative $P(\text{param} > 0 | \text{Data}, \text{Prior})$, the association with maths and gender are more positive with more than 99 percent probability after observing the data. We also get credible intervals, since we imposed priors on parameters and treat them as random we can say that with about 95 percent probability the true coefficients for Maths and Gender lie in their respective credible intervals. These should be fairly close but not the same as frequentist intervals given that our sample size is large enough that data dominates the prior. Note though that we already have the entire approximate posterior distribution of maths scores on stat scores and posterior Sd of prediction. We therefore can estimate mean risk of failing an exam and get posterior credible intervals for it by passing our simulated posterior through the normal CDF.

```
#Predict Scores for the Sample#
sigma_draws <- as.data.frame(BayesReg)$sigma
risk_table <- data.frame(
  Adv.Maths = Maths_and_Stats_grades$Adv.Maths
)
pointpredictions <- posterior_epred(BayesReg,newdata=Maths_and_Stats_grades)
#Set up the CDF for P(Score<40) and apply to predictions and sampled sds#
risk_grid_draws <- pnorm(40, mean = pointpredictions, sd = sigma_draws)
risk_table$Risk_Mean <- colMeans(risk_grid_draws)
risk_table$Risk_Lower <- apply(risk_grid_draws, 2, quantile, probs = 0.025)
risk_table$Risk_Upper <- apply(risk_grid_draws, 2, quantile, probs = 0.975)
ggplot(data =risk_table,mapping = aes(x=Adv.Maths,y=Risk_Mean,colour = Maths_and_Stats_grades$Gender,fill=Maths_and_Stats_grades$Gender))+geom_jitter()+geom_line(linewidth = 1)+ geom_ribbon(
  aes(ymin = Risk_Lower, ymax = Risk_Upper), alpha = 0.3, colour = NA) +
  scale_y_continuous(labels = scales::percent_format(),breaks = scales::breaks_pretty(n = 10))
+
  labs(
    x = "Advanced Maths Score",
    y = "Probability of Failing (< 40)",
    title = "Risk of Failing Advanced Stats",
    subtitle = "Shaded regions represent 95% posterior credible intervals"
  ) +
  theme_minimal()
```

Risk of Failing Advanced Stats

Shaded regions represent 95% posterior credible intervals



Unlike the frequentist model we can now say that if you are female and want to achieve 10 percent risk of failing stats- then to be 95 percent sure that the actual risk isn't above 10 you need to have achieved a minimum of 67 in Maths. Or if you are a male and achieved 40 in Maths your risk of failing stats is between 22 and 32 percent with 95 percent confidence. And all this you can achieve by just feeding the baysian model we already estimated to the normal CDF, isn't that cool. And you also get a personal risk assessment tool as bonus!